

UNIT V

Automation and Testing Tools: need for automation, categorization of testing tools, selection of testing tools, Cost incurred, Guidelines for automated testing, overview of some commercial testing tools such as Win Runner, Load Runner, Jmeter and JUnit . Test Automation using Selenium tool.

Testing Object Oriented Software: basics, Object oriented testing Testing

Web based Systems: Challenges in testing for web based software, quality aspects, web engineering, testing of web based systems, Testing mobile systems.

Automation and Testing Tools

Automation and Testing Tools: need for automation, categorization of testing tools, selection of testing tools, Cost incurred, Guidelines for automated testing, overview of some commercial testing tools such as Win Runner, Load Runner, Jmeter and JUnit . Test Automation using Selenium tool.

Automation Testing is a software testing technique that performs using special automated testing software tools to execute a test case suite.

Manual Testing is performed by a human sitting in front of a computer carefully executing the test steps. The automation testing software can also enter test data into the System Under Test, compare expected and actual results and generate detailed test reports.

1.Need For Automation:

1. Reduction of testing Effort
2. Reduces the testers' involvement in executing tests
3. Facilitates Regression Testing
4. Avoids Human mistakes
5. Reduces overall cost of the software
6. Simulated testing
7. Internal Testing
8. Test case Design

The following benefits of automation are

1. **Reduction of testing effort:**
 - Test cases for a complete software may be hundreds of thousands or more in number.
 - Executing all of them manually takes a lot of testing effort and time.
 - Execution of test suits through software tools greatly reduces the amount of time required.
2. **Reduces the testers' involvement in executing tests:**
 - Sometimes executing the test cases takes a long time.
 - Automating this process of executing the test suit will relieve the testers to do some other work, thereby increasing the parallelism in testing efforts.
3. **Facilitates regression testing:**
 - Regression testing is the most time consuming process.
 - Automate the process of regression testing, then testing effort as well as the time taken will reduce as compared to manual testing.
4. **Avoids human mistakes:**
 - Manually executing the test cases may incorporate errors in the process or sometimes, biased towards limited test cases while checking the software.
 - Testing tools will not cause these problems which are introduced due to manual testing.

5. Reduces overall cost of the software:

- If testing time increases, cost of the software also increases.
- But due to testing tools, time and therefore cost can be reduced to a greater level as testing tools ease the burden of the test case production and execution.

6.Simulated testing:

- Load performance testing is an example of testing where the real-life situation needs to be simulated in the test environment.
- Automated tools, on the other hand, can create millions of concurrent virtual users/data and effectively test the project in the test environment before releasing the product.

7.Internal testing:

- Testing may require testing for memory leakage or checking the coverage of testing.
- Automation tools can help in these tasks quickly and accurately, whereas doing this manually would be cumbersome, inaccurate, and time-consuming.

8.Test enablers

- While development is not complete, some modules for testing are not ready.
- At that time, stubs or drivers are needed to prepare data, simulate environment, make calls, and then verify results.

2 .CATEGORIZATION OF TESTING TOOLS

The different categories of testing tools

1. Static And Dynamic Testing Tools
2. Testing Activity Tools

These tools are based on the type of execution of test cases, namely static and dynamic.

Static testing tools

- For static testing, there are static program analysers which scan the source program and detect possible faults and anomalies.
- These static tools parse the program text, recognize the various sentences, and detect the following:
 1. Statements are well-formed. „
 2. Inferences about the control flow of the program. „
 3. Compute the set of all possible values for program data.

Static tools perform the following types of static analysis:

1. Control Flow Analysis
2. Data use Analysis
3. Interface Analysis

4. Path Analysis

1. **Control flow analysis** This analysis detects loops with multiple exits and entry points and unreachable code.
2. **Data use analysis** It detects all types of data faults.
3. **Interface analysis** It detects all interface faults. It also detects functions which are never declared and never called or function results that are never used.
4. **Path analysis** It identifies all possible paths through the program and unravels the program's control.

Dynamic testing tools

These tools support the following: „

- Dynamic testing activities. „
- Automated test tools enable the test team to capture the state of events during the execution of a program by preserving a snapshot of the conditions. These tools are sometimes called **program monitors**.

The monitors perform the following functions:

- List the number of times a component is called or line of code is executed. This information about the statement or path coverage of their test cases is used by testers.
- Report on whether a decision point has branched in all directions, thereby providing information about branch coverage.
- Report summary statistics providing a high-level view of the percentage of statements, paths, and branches that have been covered by the collective set of test cases run. This information is important when test objectives are stated in terms of coverage.

2 TESTING ACTIVITY TOOLS

These tools are based on the testing activities or tasks in a particular phase of the SDLC.

Testing activities can be categorized as :

1. Reviews and inspections „
2. Test planning „
3. Test design and development
4. „Test execution and evaluation

---**The testing tools based on these testing tasks.**

1.Tools for review and inspections:

Since these tools are for static analysis on many items, some tools are designed to work with specifications but there are far too many tools available that work exclusively with code.

In this category, the following types of tools are required:

- **Complexity analysis tools**

- It is important for testers that complexity is analysed so that testing time and resources can be estimated.
- The complexity analysis tools analyse the areas of complexity and provide indication to testers.

- **Code comprehension**

- These tools help in understanding dependencies, tracing program logic, viewing graphical representations of the program, and identifying the dead code.
- All these tasks enable the inspection team to analyse the code extensively.

2.Tools for test planning

The types of tools required for test planning are:

1. Templates for test plan documentation
2. Test schedule and staffing estimates
3. Complexity analyser

- **Tools for test design and development**

The types of tools required for test design and development.

1.Test data generator

- It automates the generation of test data based on a user defined format.
- These tools can populate a database quickly based on a set of rules, whether data is needed for functional testing, data-driven load testing, or performance testing.

2.Test case generator

- It automates the procedure of generating the test cases.
- It works with a requirement management tool which is meant to capture requirements information.
- Test case generator uses the information provided by the requirement management tool and creates the test cases.
- The test cases can also be generated with the information provided by the test engineer regarding the previous failures that have been discovered by him.
- This information is entered into this tool and it becomes the knowledge-based tool that uses the knowledge of historical figures to generate test cases.

3.Test execution and evaluation tools The types of tools required for test execution and evaluation are :

- **Capture/playback tools:**These tools record events (including keystrokes, mouse activity, and display output) at the time of running the system and place the information into a script. The tool can then replay the script to test the system.
- **Coverage analysis tools:** These tools automate the process of thoroughly testing the software and provide a quantitative measure of the coverage of the system being tested.

These tools are helpful in the following:

1. Measuring structural coverage which enables the development and test teams to gain insight into the effectiveness of tests and test suites „
2. Quantifying the complexity of the design
3. „ Help in specifying parts of the software which are not being covered „
4. Measure the number of integration tests required to qualify the design „
5. Help in producing integration tests „
6. Measuring the number of integration tests that have not been executed „
7. Measuring multiple levels of test coverage, including branch, condition, decision/condition, multiple conditions, and path coverage.

4.Memory testing tools

These tools verify that an application is properly using its memory resources.

They check whether an application is: „

1. Not releasing memory allocated to it „
2. Overwriting/overreading array bounds „
3. Reading and using uninitialized memory.

5.Test management tools:

- Test management tools try to cover most of the activities in the testing life cycle. These tools may cover planning, analysis, and design.
- Some test management tools such as Rational's TestStudio are integrated with requirement and configuration management and defect tracking tools, in order to simplify the entire testing life cycle.

6.Network-testing tools:

- There are various applications running in the clientserver environments.
- These applications pose new complexity to the testing effort and increases potential for errors due to inter-platform connectivity. Therefore, these tools monitor, measure, test, and diagnose performance across an entire network including the following: „
 - Cover the performance of the server and the network
 - Overall system performance „
 - Functionality across server, client, and the network.

7.Performance testing tools:

- There are various systems for which performance testing is a must but this becomes a tedious job in real-time systems.
- Performance testing tools help in measuring the response time and load capabilities of a system.

3. SELECTION OF TESTING TOOLS

Some guidelines to be followed by selecting a testing tool are

1. Match the tool to its appropriate use
2. Select the tool to its appropriate SDLC phase
3. Select the tool to the skill of the tester
4. Select a tool which is affordable
5. Determine how many tools are required for testing the system
6. Select the tool after examining the schedule of testing

1.Match the tool to its appropriate use:

- A tool may not be a general one or may not cover many features. Rather, most of the tools are meant for specific tasks.
- Therefore, the tester needs to be familiar with both the tool and its uses in order to make a proper selection.

2.Select the tool to its appropriate SDLC phase:

- Since the methods of testing changes according to the SDLC phase, the testing tools also change.
- Therefore, it is necessary to choose the tool according to the SDLC phase, in which testing is to be done.

3.Select the tool to the skill of the tester

- The individual performing the test must select a tool that conforms to his skill level.
- For example, it would be inappropriate for a user to select a tool that requires programming skills when the user does not possess those skills.

4.Select a tool which is affordable :

- Tools are always costly and increase the cost of the project.
- Therefore, choose the tool which is within the budget of the project. Increasing the budget of the project for a costlier tool is not desired.
- If the tool is under utilization, then added cost will have no benefits to the project. Thus, once you are sure that a particular tool will really help the project, then only go for it otherwise it can be managed without a tool also.

5.Determine how many tools are required for testing the system:

- A single tool generally cannot satisfy all test requirements. It may be possible that many test tools are required for the entire project.
- Therefore, assess the tool as per the test requirements and determine the number and type of tools required.

6.Select the tool after examining the schedule of testing:

- First, get an idea of the entire schedule of testing activities and then decide whether there is enough time for learning the testing tool and then performing automation with that tool.

- If there is not enough time to provide training on the tool, then there is no use of automation.

4. COSTS INCURRED IN TESTING TOOLS

Automation is not free. Obviously employing the testing tools incur a high cost.

Following are some facts pertaining to the cost incurred in testing tools

1. Automated script development:

- Automated test tools do not create test scripts.
- Therefore, a significant time is needed to program the tests. Scripts are themselves programming languages.
- automating test execution requires programming exercises.

2. Training is required:

- It is not necessary that the tester will be aware of all the tools and can use them directly.
- Require training regarding the tool, otherwise it ends up on the shelf or implemented inefficiently.
- Therefore, it becomes necessary that in a new project, cost of training on the tools should also be included in the project budget and schedule.

3. Configuration management :

Configuration management is necessary to track large number of files and test related artifacts.

4. Learning curve for the tools:

- There is a learning curve in using any new tool.
- For example, test scripts generated by the tool during recording must be modified manually, requiring tool-scripting knowledge in order to make the script robust, reusable, and maintainable.

5. Testing tools can be intrusive:

- It may be necessary that for automation some tools require that a special code is inserted in the system to work correctly and to be integrated with the testing tools. These tools are known as intrusive tools which require addition of a piece of code in the existing software system.
- Intrusive tools pose the risk that defects introduced by the code inserted specifically to facilitate testing could interfere with the normal functioning of the system.
- It may be possible that your requirement is not satisfied with just one tool for automation. In such a case, you have to go for many tools which incur a lot of cost.

5. GUIDELINES FOR AUTOMATED TESTING

Automation is not a magical answer to the testing problem. It is necessary that you carefully plan the automation before adopting it. Decide which tool and how many tools are required, how much resources are required including the cost of the tool and the time spent on training.

The guidelines to be followed planned for automation in testing.

1.Consider building a tool instead of buying one, if possible

- The requirement is small and sufficient resources allow, then go for building the tool instead of buying,
- Whether to buy or build a tool requires management commitment, including budget and resource approvals.

2.Test the tool on an application prototype

- While purchasing the tool, it is important to verify that it works properly with the system being developed.
- It is not possible as the system being developed is often not available.
- Therefore, it is suggested that if possible, the development team can build a system prototype for evaluating the testing tool.

3.Not all the tests should be automated

- Automated testing is an enhancement of manual testing, but it cannot be expected that all test on a project can be automated.
- It is important to decide which parts need automation before going for tools.
- Some tests are impossible to automate, such as verifying a printout. It has to be done manually.

4.Select the tools according to organizational needs

- Do not buy the tools just for their popularity or to compete with other organizations.
- Focus on the needs of the organization and know the resources (budget, schedule) before choosing the automation tool.

5.Use proven test-script development techniques

Automation can be effective if proven techniques are used to produce efficient, maintainable, and reusable test scripts.

The following are some hints:

- 1.Read the data values from either spreadsheets or tool-provided data pools, rather than being hard-coded into the test-case script because this prevent test cases from being reused. Hard-coded values should be replaced with variables and whenever possible read data from external sources.
2. Use modular script development. It increases maintainability and readability of the source code.
3. Build library of reusable functions by separating the common actions into shared script library usable by all test engineers.
4. All test scripts should be stored in a version control tool.

6. Automate the regression tests whenever feasible

- Regression testing consumes a lot of time. If tools are used for this testing, the testing time can be reduced to a greater extent.
- Therefore, whenever possible, automate the regression test cases.

6. OVERVIEW OF SOME COMMERCIAL TESTING TOOLS

1. Mercury Interactive's WinRunner

- It is a tool used for performing functional / regression testing.
- It automatically creates the test scripts by recording the user interactions on GUI of the software.
- These scripts can be run repeatedly whenever needed without any manual intervention.
- The test scripts can also be modified if required because there is support of **Test Script language (TSL)** with a 'C' like syntax.
- There is also provision for bringing the application to a known state if any problem has occurred during automated testing.
- WinRunner executes the statements by default with an interleaving of one second.
- But if some activities take more time to complete, then it synchronizes the next test cases automatically by waiting for the current operations to be completed.
- Main Features of Win Runner are:
 - Developed by Mercury Interactive
 - Functionality testing tool
 - Supports C/s and web technologies such as (VB, VC++, D2K, Java, HTML, Power Builder, Delphe, Cibell (ERP))
 - To Support .net, xml, SAP, People soft, Oracle applications, Multimedia we can use QTP.
 - Winrunner run on Windows only.
 - Tool developed in C on VC++ environment.

2. Segue Software's SilkTest

- This tool is also for functional/regression testing.
- It supports 4Test as a scripting language which is an object-oriented scripting language.
- SilkTest has a provision for customized in-built recovery system which helps in continuing the automated testing even if there is some failure in between.

3. IBM Rational SQA Robot

- It is another powerful tool for functional/regression testing.
- Synchronization of test cases with a default delay of 20 seconds is also available.
- Rational Robot is an automated functional, regression testing tool for automating Windows, Java, IE and ERP applications under windows platform.
- Rational Robot provides test cases for common objects such as menus, lists, bitmaps and specialized test cases for objects specific to the development environment.

4.Mercury Interactive's LoadRunner

- This tool is used for performance and load testing of a system. Generally, the tool is helpful for client/server applications of various parameters with their actual load like response time, the number of users, etc.
- The major benefit of using this tool is that it creates virtual users on a single machine and tests the system on various parameters.
- Performance and load testing is done with minimum infrastructure.

5.Apache's JMeter.

- The Apache Jmeter application is open source software tool used for performance and load testing.
- A 100% pure Java application designed to load test functional behavior and measure performance.
- It was originally designed for testing Web Applications but has since expanded to other test functions.

6.Mercury Interactive's TestDirector

- TestDirector is a test management tool.
- It is a web-based tool with the advantage of managing the testing if two teams are at different locations. It manages the test process with four phases: specifying requirements, planning tests, running tests, and tracking defects.
- There is advantage that the tests are planned as per the requirements evolved.
- Once the test plan is ready, the test cases are executed. Defect-tracking can be done in any phase of test process.
- During defect-tracking, new bug can be reported; assign responsibility to someone for bug repair; assign priority to the bug; bug repair status can be analysed, etc.
- This tool can also be integrated with Load Runner or Win Runner.

7.Test Automation using Selenium tool.

Whenever we are talking about the open-source tools of automation testing, one name came into every automation test engineer's mind: Selenium.

- Selenium is the highly recommended and widely used functional testing tool, which is well-suited for non-functional testing tools.
- Selenium is an open-source tool which means, it does not require any licensing.
- only test the web application using the selenium tool, and the Standalone application cannot automate in Selenium.
- It is most commonly used to implement Functional test scripts.
- It can be combined with several automation testing tools to achieve continuous testing, for example, Maven and Jenkins.
- It can be associated with many devices such as TestNG and JUnit to manage the test cases and generate the test reports.

Characteristics of Selenium

- Following are the basic features of selenium:
- Selenium can be easily deployed on several platforms like Windows, Linux, Solaris, and MacOS.

Furthermore, it also supports operating systems for mobile applications, for example, iOS, Windows mobile, and android. Test Automation using Selenium tool.

- Selenium supports several programming languages such as C#, Java, Perl, PHP, Python, and Ruby.
- Selenium become extremely resourceful while we were implementing the test script and analyzing it across many browsers concurrently.

TESTING OBJECT ORIENTED SOFTWARE

1. Basics

2. Object Oriented Testing

- Object-oriented technology (OOT) was established with a number of object-oriented programming languages.
- OO paradigm is embedded in the complete software engineering approach.
- Software built with OO follows all the steps of **OO software engineering** (OOSE).
- Modeling, analysis, design, and testing of OO software is different as compared to the structured approach

1 .OOT BASICS

1 TERMINOLOGY

2 OBJECT-ORIENTED MODELING AND UML

- The major reason for adopting OOT and discarding the structured approach is the complexity of the software.
- Structured approach is unable to reduce the complexity as the software size increases, thereby increasing the schedule and cost of the software.

1 TERMINOLOGY

1.Object :

- ‘Objects are abstraction of entities in the real world representing a particularly dense and cohesive clustering of information.
- objects are individual entities formed from two components: state and behaviour.
- The state is modeled by a data structure.
- The behaviour is modeled by a set of operations (or actions) which manipulate the data structure to obtain the required change of state and result. Together, they form a single coherent unit.
- The behaviour is invoked by sending a message to the object requesting a particular service.

2. Class :

- All objects with the same structure and behaviour can be defined with a common blueprint.
- This blueprint called a class, defines the data and behaviour common to all objects of a certain type. Thus class is a template that describes collection of similar objects, e.g. vehicle is a class and car is one object of class vehicle.

- The operations provided by a class are also known as features, and each item of data defined by a class is a data-member.
- Any class that invokes the features of another class is known as a client of the class providing the features.

3.Encapsulation :

- A class contains data as well as operations that operate on data. In this way, data and operations are said to be encapsulated into a single entity.
- Encapsulation avoids the accidental change in data such that no one can directly access the object's data.
- For example: capsule, it is wrapped with different medicines.

4.Abstraction:

- A simplified description, or specification of a system that emphasizes some of the system's details or properties while suppressing others.
- A good abstraction is one that emphasizes details that are significant to the user and suppresses details that are immaterial to him'.
- For example: phone call, we don't know the internal processing.

5.Inheritance :

- It is a strong feature of OOT.
- It provides the ability to derive new class from existing classes.
- The derived class inherits all the data and behaviour (operations) of the original base class.
- Moreover, derived class can also add new features
- e.g. employee is the base class and we can derive manager as derived class.

6.Polymorphism :

- This feature is required to extend an existing OO system.
- Polymorphism means having many forms and implementations of a particular functionality.
- For example, when a user wants to draw some shape (line, circle, etc.), the system issues a draw command.
- The system is composed of many shapes, each of them having the behaviour to draw itself.
- So when the user wants to draw a circle, the circle object's draw command is invoked.
- By using polymorphism, the system figures out, as it is running, which type of shape is being drawn .

2 OBJECT-ORIENTED MODELING AND UML

- Modeling is the method to visualize, specify different parts of a system, and the relationship between them. Modeling basically divides the complex problem into smaller understandable elements and provides a solution to the problem.
- In object-oriented development, Unified modeling language (UML) has become the de facto standard for analysis and design of OOS.
- UML provides a number of graphical tools that can be used to visualize a system from different viewpoints. The views generally defined in it are:

1.User view:

- This view is from the user's viewpoint which represents the goals and objectives of the system as perceived by the user.
- It includes use-cases and use-case diagram as a tool for specifying the objectives.

2.Structural view:

- This view represents the static view of the system representing the elements that are either conceptual or physical.
- Class diagrams and object diagrams are used to represent this view.

3.Behavioural view:

This view represents the dynamic part of the system representing behaviour over time and space.

The tools used for behavioral view are:

- collaboration diagrams
- sequence diagrams
- state chart diagrams
- activity diagrams.

4.Implementation view:

This view represents the distribution of logical elements of the system. It uses component diagrams as a tool.

5.Environmental view :

- This view represents the distribution of physical elements of the system.
- It depicts nodes which are part of the physical hardware for deployment of the system.
- It uses deployment diagram as a tool.
- All the graphical tools used in the UML modeling.
- It is necessary to understand them as you need to verify and validate the OO software development through these models.
- Use-case model
- Requirements for a system can be elicited and analysed with the help of a set of scenarios that identify how to use the system.

The following components are used for this model:

1.Actors :

- An actor is an external entity or agent that provides or receives information from the system.
- Formally, an actor is anything that communicates with the system or product and that is external to the system itself.

2.Use-case:

- A use-case is a scenario that identifies and describes how the system will be used in a given situation.
- Each use-case describes an unambiguous scenario of interactions between actors and the system.
- Thus, use-cases define functional and operational requirements of the system.

- A complete set of use-cases describe all the means to use a system.
- Use cases can be represented as use-case templates and use-case diagrams.

3.Use-case templates :

They are prepared as structured narrative mentioning the purpose of use-case, actors involved, and the flow of events.

4.Use-case diagrams These show the visual interaction between use-cases and actors.

5. Class diagrams

- A class diagram represents the static structure of the system using classes identified in the system with associations between them.
- It shows the interactions between the classes in the system.

6.Object diagrams :

- An object diagram represents the static structure of the system at a particular instance of time.
- It shows a set of objects and their relationships at a point of time.

7.Class-responsibility-collaboration (CRC) diagrams:

- This diagram represents a set of classes and the interaction between them using associations and messages sent and received by the classes.
- These diagrams are used to understand the messages exchanged between the classes.

CRC model index card This card represents a class.

The card is divided into three sections:

- On the top of the card, name of class is mentioned.
- In the body of the card on the left side, responsibilities of the class are mentioned.
- In the body of the card on the right side, the collaborators (relationship between classes) are mentioned.

8.Sequence diagram:

- It is a pictorial representation of all possible sequences along the time-line axis to show the functionality of one use-case
- e.g. for the use-case Deposit the money in a bank, there are sequences of events—customer goes to cashier with cash and form, submits it to the cashier, cashier receives the cash and form, deposits the amount and updates the balance, returns receipt to the customer.

9.State chart diagrams:

- These diagrams represent the states of reactive objects (which respond to external events) and transitions between the states.
- A state chart depicts the object's initial state where it performs some activity and transitions after which the object enters another state.

- Thus, these diagrams represent the behaviour of objects with respect to their states and transitions.

10.Activity diagrams:

It is a special case of a state chart diagram, wherein states are activity states and transitions are triggered by the completion of activities.

11.Component diagrams :

At the physical level, the system can be represented as components. The components can be in the form of packages or sub-systems containing classes. The component diagrams show the set of components and their relationships.

12.Deployment diagrams :

- These diagrams are also at the physical level. After the components and their interaction have been identified in the component diagram, the components are deployed for execution in the actual hardware nodes identified according to the implementation.
- Thus, these diagrams depict the mappings of components to the nodes involved in the physical implementation of a system.

2 .OBJECT-ORIENTED TESTING

1 CONVENTIONAL TESTING AND OOT

2 OBJECT-ORIENTED TESTING AND MAINTENANCE PROBLEMS

3 ISSUES IN OO TESTING

4 STRATEGY AND TACTICS OF TESTING OOS

5 VERIFICATION OF OOS

6 VALIDATION ACTIVITIES

7 TESTING OF OO CLASSES

8 INHERITANCE TESTING

9 INTEGRATION TESTING

10 UML-BASED OO TESTING

1 CONVENTIONAL TESTING AND OOT:

- The difference between unit testing for conventional software and object-oriented software arises from the nature of classes and objects.
- unit testing only addresses the testing of individual operations and data.
- This is only a subset of the unit testing for object-oriented software, as the meaning and behaviour of the resources encapsulated in the classes depend on other resources with which they are encapsulated.
- Integration testing (for conventional software) is truly the unit testing of object-oriented software.

- The object-oriented paradigm lowers the complexity and hence, lessens the need for basis path testing.

2 OBJECT-ORIENTED TESTING AND MAINTENANCE PROBLEMS

some of these problems which are discussed here:

1.Understanding problem

- The encapsulation and information hiding features are not easily understood the very first time.
- There may be a chain of invocations of member functions due to these features.
- A tester needs to understand the sequence of this chain of invocations and design the test cases accordingly.

2.Dependency problem

- There are many concepts in OO that gives rise to the dependency problem.
- The dependency problem is mainly due to complex relationships,
- e.g. inheritance, aggregation, association, template class instantiation, class nesting, dynamic object creation, member function invocation, polymorphism, and dynamic binding relationships.
- These relationships imply that one class inevitably depends on another class.
- For example, the inheritance relationship implies that a derived class reuses both the data members and the function members of a base class; and hence, it is dependent on the base class.

The complex relationships that exist in an OOS make testing and maintenance extremely difficult in the following way:

(i) Dependency of the classes increases the complexity of understanding and hence testing.

(ii) In OO software, the stub development is more difficult as it needs to understand the chain of called functions, possibly create and properly initialize certain objects, and write their code.

(iii) Template class feature is also difficult to test as it is impossible to predict all the possible uses of a template class and to test them.

(iv) Polymorphism and dynamic binding features add to the challenge of testing them as it is difficult to identify and test the effect of these features.

(v) As the dependencies increase in OOS, it becomes difficult to trace the impact of changes, increasing the chances of bugs, and hence making testing difficult.

3.State behaviour problem

- Objects have states and state-dependent behaviours .
- That is, the effect of an operation on an object also depends on the state of the object and may change its state. Thus, combined effect of the operations must be tested.

3 ISSUES IN OO TESTING

Based on the problems discussed for testing an OO software

The following are some issues which come up while performing testing on OO software.

1.Basic unit for testing

- There is nearly universal agreement that class is the natural unit for test case design.
- Other units for testing are aggregations of classes: class clusters, and application systems.
- The intended use of a class implies different test requirements,
- e.g., application-specific vs general-purpose classes, abstract classes, and parameterized (template) classes. Testing a class instance (an object) can verify a class in isolation.
- However, when verified classes are used to create objects in an application system, the entire system must be tested as a whole before it can be considered verified.

2.Implications of inheritance

- In case of inheritance, the inherited features require retesting because these features are now in, new context of usage.
- Therefore, while planning to test inheritance, include all inherited features. Moreover, planning should also consider the multiple contexts of usage in case of multiple inheritances.

3.Polymorphism

- In case of polymorphism, each possible binding of a polymorphic component requires a separate test to validate all of them.
- However, it may be hard to find all such bindings.
- This again increases the chance of errors and poses an obstacle in reaching the coverage goals.

4.White-box testing

There is also an issue regarding the testing techniques for OO software.

The issue is that the conventional white-box techniques cannot be adapted for OO software.

In OO software, class is the major component to be tested, but conventional white-box testing techniques cannot be applied on testing a class.

These techniques may be applicable to individual methods but not suitable for OO classes.

5.Black-box testing

- The black-box testing techniques may be suitable to OO software for conventional software, but sometimes re-testing of inherited features require examination of class structure.
- This problem also limits the black box methods for OO software.

6.Integration strategies

- Integration testing is another issue which requires attention and there is need to devise newer methods for it. However, there is no obvious hierarchy of methods or classes.
- Therefore, conventional methods of incremental integration testing cannot be adopted.

4 STRATEGY AND TACTICS OF TESTING OOS

The strategy for testing OOS is the same as the one adopted earlier for procedural software, i.e. verification and validation.

However, tactics for testing may differ depending upon the structure of OOS. Object-oriented software testing is generally done bottom-up at four levels:

1. Method-level testing

It refers to the internal testing of an individual method in a class. Existing testing techniques for conventional programming languages are fully applicable to method-level testing. Class-level testing Methods and attributes make up a class.

2. Class-level (or intra-class) testing refers to the testing of interactions among the components of an individual class.

3. Cluster-level testing Cooperating classes of objects make up a cluster. Cluster level (or inter-class) testing refers to the testing of interactions among objects.

4. System-level testing Clusters make up a system. System-level testing is concerned with the external inputs and outputs visible to the users of a system.

5 VERIFICATION OF OOS(Object-oriented software)

- The verification process is different in analysis and design steps.
- OOS is largely dependent on OOA and OOD models, it becomes important to verify them.
- For example, if there is problem in the definition of class attributes and is uncovered during analysis, then this problem will propagate in design and other phases too.
- OOA is not matured enough in one iteration only. So it requires many iterations for a reliable analysis model.
- Verification can play an important role in these iterations of OOA.
- Each iteration should be verified so that we know that the analysis is complete.
- For example, consider a class in which a number of attributes are defined during the first iteration, and an attribute may be appended in another iteration due to some misunderstanding.
- If there is verification of OOA at each iteration, then this error may be pointed out in a review and thereby avoid problems.

Thus, all OO models must be tested for their correctness, completeness, etc.

Verification of OOA and OOD models :

1. Assess the class-responsibility-collaborator (CRC) model and object relationship diagram.
2. Review system design (examine the object-behaviour model to check mapping of system behaviour to subsystems, review concurrency and task allocation, use use-case scenarios to exercise user interface design).
3. Test object model against the object relationship network to ensure that all design objects contain necessary attributes and operations needed to implement the collaborations defined for each CRC card.

4. Review detailed specifications of algorithms used to implement operations using conventional inspection techniques.

Verification points of some of the models are given below:

1. Check that each model is correct in the sense that proper symbology and modeling conventions have been used.
2. Check that each use-case is feasible in the system and unambiguous such that it should not repeat operation.
3. Check that a specific use-case needs modification in the sense that it should be divisible further in more use-cases.
4. Check that actors identified in use-case model are able to cover the functionality of the whole system. If any new actor is needed to fulfill the requirement, then add it.
5. All interfaces between actors and use-cases in use-case diagram must be continuously checked such that there should be no inconsistency.
6. All events and their sequences must be checked iteratively in a sequence diagram. If any detail of the operation is missing, then events can be added or deleted if required.
7. Collaboration diagrams must be verified for distribution of processing between objects. Suppose the collaboration diagram was shaped like a star, with several objects communicating with a central object. Analyst may conclude that the system is too dependent on the central object and redesign the objects to distinguish the processing power more evenly .
8. Each identified class must be checked against defined selection characteristics such that it should be unique and unambiguous.
9. All collaborations in class diagrams must be verified continuously such that there should be no inconsistency.
10. Check the description of each CRC index card to determine the delegated responsibility.

6 VALIDATION ACTIVITIES

All validation activities described earlier are also implemented in OOS.

But the meaning of unit and integration testing changes due to the nature of OOS. Therefore, new methods or techniques are required to test all the features of OOS.

1. Unit/Class Testing

Unit or module testing has no meaning in the context of OOS. Since the structure of OOS is not based on action, module testing has lost its meaning.

OOS is based on the concept of class and objects.

The smallest unit to be tested is a class. The nature of OOS is different as compared to procedural software; therefore many issues rise for testing a class.

Issues in testing a class :

- A class cannot be tested directly. First, we need to create the instances of a class as object and then test it. Thus, an abstract class cannot be tested, as its objects cannot be created.

- Operations or methods in a class are the same as functions in traditional procedural software. One idea is that they can be tested independently using the previous white-box or black-box methods. But in the context of OOS, this isolated testing has no meaning. Thus the methods should also be tested with reference to a class or class hierarchy in the direction of complete testing of a class.

One important issue in class testing is the testing of inheritance. If some classes have been derived from a base class, then independent base class testing is not sufficient. All derived classes in the hierarchy must also be tested depending on the kind of inheritance.

The following special cases also apply to inheritance testing:

(a) Abstract classes can only be tested indirectly (because it can't be instantiated). Hence, abstract classes have to be tested by testing instance of its descendant specialized classes.

(b) Special tests have to be written to test the inheritance mechanism, e.g. a child class can inherit from a parent and that we can create instance of the child class.

- There may be operations in a class that change the state of the object on their execution. Thus, these operations should be tested with reference to state change of object.
- Polymorphism provides us with elegant compact code, but it can also make testing more complicated. The identification of all the paths through a section of code that uses polymorphism is harder because the criteria which indicate different paths are not specifically mentioned in the code.

7 TESTING OF OO CLASSES

Feature-based Testing of Classes

- There are a number of code complexity metrics available which vary from simple ones such as the number of the source lines of code, to the cyclomatic complexity metric.
- These metrics can be used to measure the complexity of the individual features of a class providing the tester with an accurate way of determining which features to consider first for testing.

The features of a class can be categorized into six main groups :

Create These are also known as **constructors**. These features perform the initial memory allocation to the object, and initialize it to a known state.

Destroy These are also known as **destructors**. These features perform the final memory de-allocation when the object is no longer required.

Modifiers The features in this category alter the current state of the object, or any part thereof.

Predicates The features in this category test the current state of the object for a specific instance. Usually, they return a BOOLEAN value.

Selectors The features in this category examine and return the current state of the object, or any part thereof.

Iterators The features in this group are used to allow all required sub-objects to be visited in a defined order.

1. Testing feature groups

While testing the classes with features as discussed above, there may be some preferences of these feature groups according to which one is tested first.

The following guidelines for the preferred features to be tested:

- (i) The **create** features of a class must be verified first, as these features are for the initialization of every object created. Therefore, they must be tested rigorously.
- (ii) The **selector** features can be the next features to be verified as they do not alter the state of the object. To test them, alter the state of the object either by using the appropriate constructor (if available), or by directly altering the state of the object.
- (iii) The **predicate** features of a class test for specific states of the object and therefore while constructing the test cases, care must be taken to place the object in the specific state for the required test result.
- (iv) The **modifier** features of a class are used to alter the state of the object from one valid state to another. To detect if the transition was to the correct state, the selectors and predicate features should be used where possible.
- (v) The **destructor** feature of a class performs any 'housekeeping', for example, returning any allocated memory to the heap. To test them, features which alter the state of the object and allocate memory are used to create the scenarios for the test cases, then the destructor is called

2. Role of Invariants in Class Testing

To specify the valid states and invalid states for an object, it can help in testing the classes.

The language Eiffel provides the facility of specifying the invariants. But, by providing the features for testing the validity of the object's current state, also achieve this ability.

The importance of invariants in class testing.

For example, take a class to implement an integer Stack as:

Class Stack with following data members:

Item Count, empty, full, push, pop, max_size

Here in this class, invariants can be defined as given below:

1. Item Count cannot have a negative value.
2. There is an upper bound check on Item Count, i.e. Item Count $\leq$ max size
3. Empty stack, i.e. Item Count = 0
4. Full Stack, i.e. Item Count = max_size

These invariants allow the validity of the current state of the object to be tested easily.

An invariant is also useful for ensuring that the data representation is in a valid state, even if it has been directly altered. This can help maintain the integrity of the class.

3.State-Based Testing

State-based testing can also be adopted for testing of object-oriented programs.

The systems can be represented naturally with finite state machines as states correspond to certain values of the attributes and transitions correspond to methods.

The features of a class provide the desired behaviour by interacting with the data-representation. Although these interactions are found in programs written in traditional languages, they are more visible and prevalent in object-oriented programming languages.

State-based testing tests these interactions by monitoring the effects that features have on the state of an object. The state can be determined by the values stored in each of the variables that constitute its data representation.

Each separate variable is known as a sub state, and the values it may store are known as sub state-values. The restrictions on objects that are to be represented as state machines include :

The behaviour of the object can be defined with a finite number of states.

There are a finite number of transitions between the states.

All states are reachable from the initial state.

Each state is uniquely identifiable through a set of methods.

Process of state-based testing

The process of using state-based testing techniques is as follows:

1. Defines the states.
2. Define the transitions between states.
3. Define test scenarios.
4. Define test values for each state.

consider the Stack example again.

1.Define states

The first step in using state-based testing is to define the states.

create a state model with the following states in the Stack example:

1. Initial: before creation
2. Empty: Item Count = 0

3. Holding: Item Count > 0, but less than the max capacity

4. Full: Item Count = max

5. Final: after destruction

2. Define transitions between states

The next step is to define the possible transitions between states and to determine what triggers a transition from one state to another. Generally, when we test a class, a transition is triggered when a method is invoked.

some possible transitions could be defined as follows:

1. Initial $\mathcal{A}$ Empty: Action = “create” e.g. “s = new Stack()” in Java

2. Empty $\mathcal{A}$ Holding: Action = “add”

3. Empty $\mathcal{A}$ Full:

Action = “add” if max_size = 1

4. Empty $\mathcal{A}$ Final: Action = “destroy” e.g. destructor call in C++, garbage collection in Java

5. Holding $\mathcal{A}$ Empty: Action = “delete” Thus, for each identified state,

list the following:

1. Valid actions (transitions) and their effects, i.e., whether or not there is a change of state.

2. Invalid actions that should produce an exception.

3. Define test scenario

Tests generally consist of scenarios that exercise the object along a given path through the state machine.

The following are some guidelines:

1. Cover all identified states at least once.

2. Cover all valid transitions at least once.

3. Trigger all invalid transitions at least once.

Test cases are designed such that they exercise operations that change state and those that do not change state separately.

For instance,

Test case: Create-Add-Add-Delete-Add-delete-destroy

It is also needed to check the state of the object that is being tested throughout the scenario to ensure that the theoretical state model you have defined is actually the one implemented by the class you are testing.

4. Define test values for each state

- Finally, choose test values for each individual state.
- The consideration in choosing the values is that choose unique test values and do not reuse values that were used previously in the context of other test cases.
- This strategy provides more diversity in the test suite and increases the likelihood of detecting bugs.

8 INHERITANCE TESTING

1. Super class modifications

When a method is changed in super class,

The following should be considered:

- (i) The changed method and all its interactions with changed and unchanged methods must be retested in the super class.
- (ii) The method must be retested in all the subclasses inheriting the method as extension.
- (iii) The 'super' references to the method must be retested in subclasses.

2. Inherited methods

The following inherited methods should be considered for testing:

- (i) **Extension:** It is the inclusion of super class features in a subclass, inheriting method implementation and interface (name and arguments).
- (ii) **Overriding:** It is the new implementation of a subclass method, with the same inherited interface as that of a super class method.

3. Reusing of test suite of super class

It may not be possible to reuse super class tests for extended and overridden methods due to the following reasons:

- (i) Incorrect initialization of super class attributes by the subclass.
- (ii) Missing overriding methods.
- (iii) Direct access to super class fields from the subclass code can create a subtle side.
- (iv) A subclass may violate an invariant from the super class or create an invalid state.

4. Addition of subclass method

When an entirely new method is added to a specialized subclass,

the following should be considered:

- (i) The new method must be tested with method-specific test cases.
- (ii) Interactions between the new method and the old methods must be tested with new test cases.

5.Change to an abstract super class interface

‘Abstract’ super class is the class wherein some of the methods have been left without implementation; just the interface has been defined. In this case, all the subclasses must be retested, even if they have not changed.

6.Interaction among methods

Exercise interaction among methods of all super classes and of the subclass under test? Thus, inherited methods should be retested in the context of a subclass.

Example: If we change some method $m()$ in a super class, we need to retest $m()$ inside all subclasses that inherit it.

Example: If we add or change a subclass, we need to retest all methods inherited from a super class in the context of the new/changed subclass.

7. Inheritance of Invariants of Base Class

- If an invariant is declared for a base class, the invariant may be extended by the derived class. Additional clauses may be added for new data-members declared by the derived class. However, the data-representation of the parent class still complies with the original invariant and therefore does not require retesting.
- If the invariant provided by the base class is retracted, then the base class must be retested with respect to the new invariant.
- A retraction involves the loosening of clauses, allowing the data members of the parent class to accept states that were previously not allowed.
- The retesting can be performed by re-executing all the original test cases, along with some additional test cases to cover the widened invariant. By clever analysis of the widened invariant, it may be possible to reduce the number of test cases from the original suite of test cases that need to be re-executed.

8.Incremental Testing

The technique first defines inheritance with an incremental hierarchy structure. Then this approach is used to test the inheritance of classes.

This technique is based on the approach of a modifier to define a derived class.

A modifier is first applied on the base class that alters its attributes.

The modifier and the base class, along with the inheritance rules for the language, are used to define the derived class.

the composition operator  to represent this uniting of base class and modifier, and we get $D = B \approx M$ where D is derived class, B is base class, and M is a modifier.

9 Integration Testing

Different classifications of testing levels have been proposed with the following terminology:

1.Inter-class testing This type of testing includes testing of any set of cooperating classes, aimed at verifying that the interaction between them is correct. There are no constraints on how these classes are selected.

Cluster testing This type of testing includes testing of the interactions between the different classes belonging to a subset of the system, having some specific properties called a cluster. Usually, a cluster is composed of cooperating classes providing particular functionalities (e.g., all the classes which can be used to access the file-system, or the classes composing a Java package). Clusters should provide a well-defined interface, i.e. their interfaces should be well-understood and they should mutually interact only by means of their interfaces.

Inter-cluster testing This type of testing includes testing of the interactions between already tested clusters. The result of the integration of all clusters is the whole system.

2.Thread-based Integration Testing consider object-oriented systems as sets of cooperating entities exchanging messages, threads can be naturally identified with sequences of subsequent message invocations.

Therefore, a thread can be seen as a scenario of normal usage of an object-oriented system.

Testing a thread implies testing the interactions between classes according to a specific sequence of method invocations.

A similar approach is the notion of method-message path (MM-path), defined as a sequence of method executions linked by messages.

For each identified MM-path, integration is performed by pulling together classes involved in the path and exercising the corresponding message sequence.

Thus, integration level thread is a MM-path. More precisely, J

identify two different levels for integration testing:

Message quiescence This level involves testing a method together with all methods it invokes, either directly or transitively.

Event quiescence This level is analogous to the message quiescence level, with the difference that it is driven by system-level events.

Testing at this level means exercising message chains (threads), such that the invocation of the first message of the chain is generated at the system interface level (i.e., the user interface) and, analogously, the invocation of the last message results in an event which can be observed at the system interface level. An end-to-end thread is called an atomic system function (ASF). Therefore, a system-level thread is a sequence of ASFs. ASFs have port events as their inputs and outputs, and thus a sequence of ASFs implies an interleaved sequence of port input and output events. The main drawback of this method is the difficulty in the identification of ASFs, which requires either the understanding of the whole system or an analysis of the source code.

3. Implicit Control Flow-based Integration Testing

- The integration testing should also consider the methods invocations where the call is not obvious in the source code.
- For example, the C++ expression “x*y” for object x and y.
- In this case, there is a call to a method via operator overloading for the multiplication of two objects. Thus, these types of method invocations also need to be considered in integration testing. This is known as implicit control flow-based integration testing.
- There are three approaches for this type of integration: optimistic, pessimistic, and balanced.
- To understand all three approaches, example as shown in Fig. 16.3. This example illustrates the very common example of object-oriented feature where we want to calculate the salary of employees of many categories,
- e.g. worker, assistant, and manager using the common method Cal Salary(). The Cal Salary() will be run for which employee is known at run time, i.e. it is a dynamically bound method.
- In this example, assume that employee is a base class; worker, assistant, and manager are three derived classes from employee.
- Each derived class uses the method Cal salary() with different computations. But at run time, it will be decided which Cal Salary() method is invoked.
- Thus, method invocations are not obvious. In this case, we will use the following three integration approaches as discussed below.

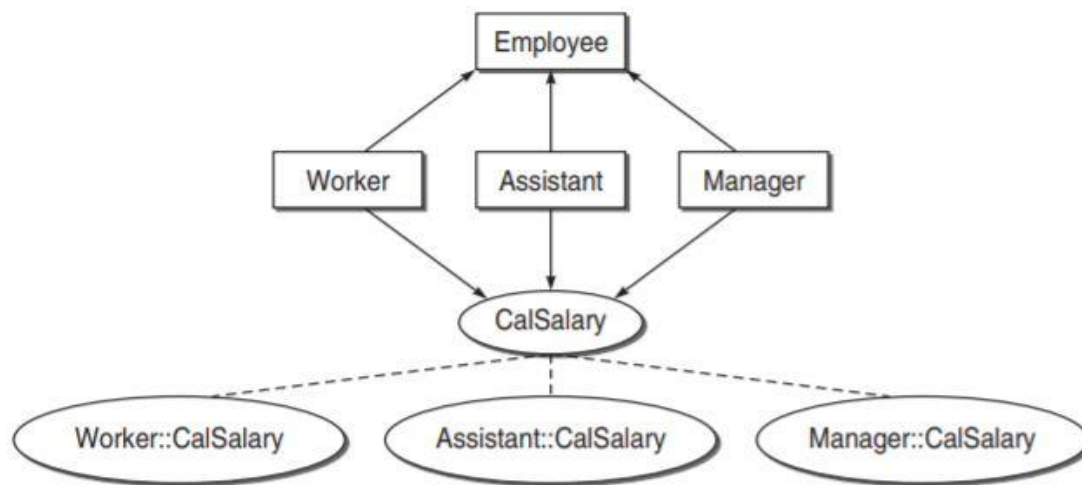


Figure 16.3 Implicit control flow example

Optimistic approach This approach is based on the optimistic view that method invocation will be confined to the level of abstraction of the source code only. Therefore, in this type of integration we will test only those method invocation or interfaces which support this abstraction as shown in Fig. 16.4.

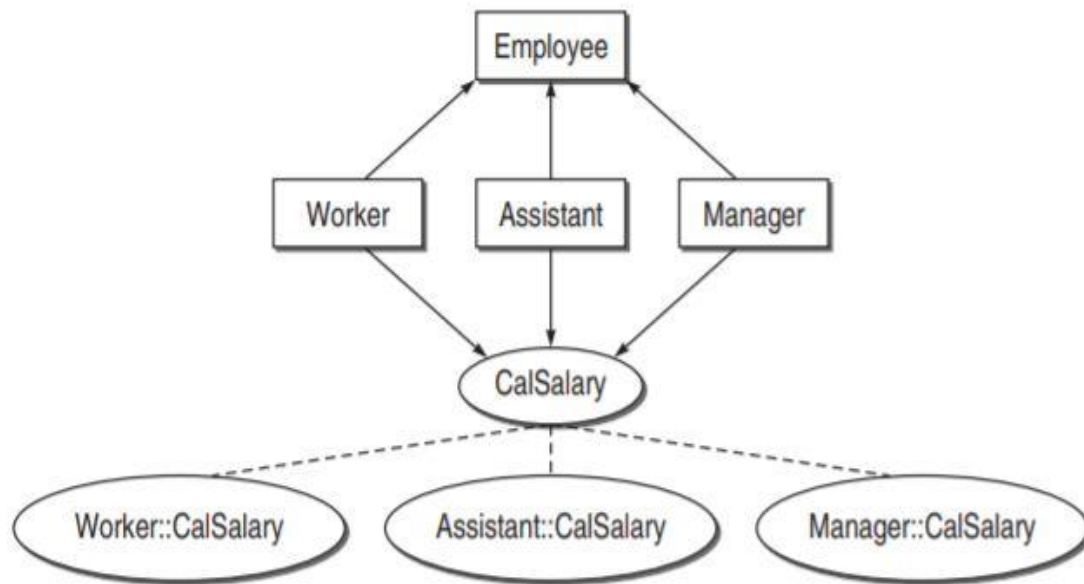


Figure 16.3 Implicit control flow example

Pessimistic approach This approach assumes that every class can call the common method Cal Salary. Therefore, all interfaces of calling the method are tested here as shown in Fig. 16.5. However, the danger in this type of integration testing is the increase in the complexity in testing.

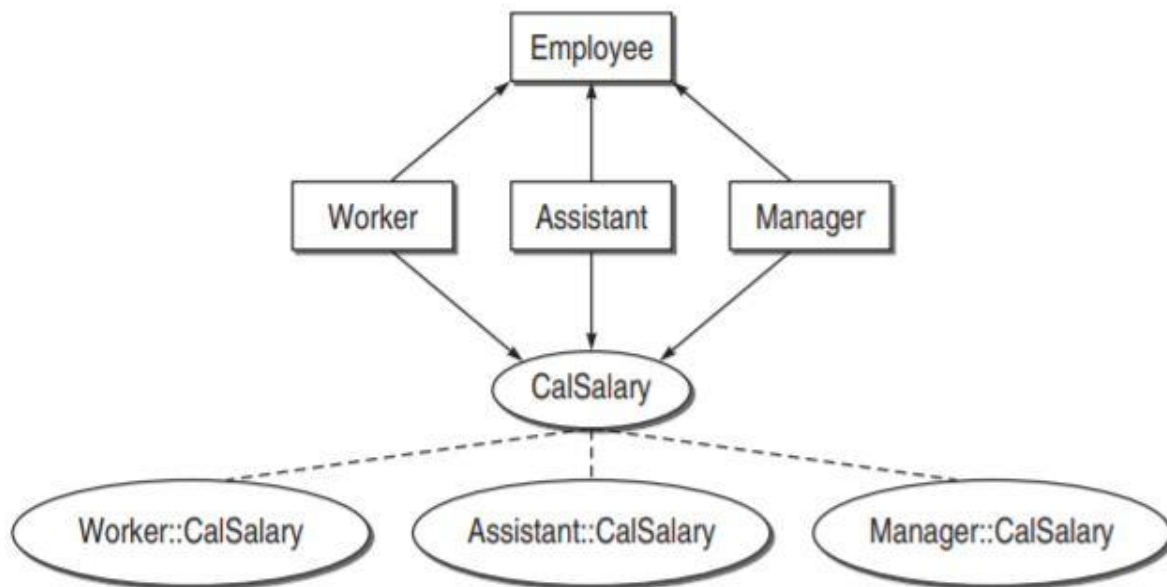


Figure 16.3 Implicit control flow example

Balanced approach This approach is a balance between the above two approaches. We must take the interfaces according to the code as well as expected method invocations. But, here we take a mid approach that all combinations of interfaces in the pessimistic approach need not be tested. Instead, only one or two classes should test source and some combinations of interfaces will be tested such that other interfaces are

likely to work properly. Therefore, we can make an equivalence set of interfaces which will be tested and others will be assumed to be working correctly as shown in Fig. 16.6. 16.2.

10 UML-BASED OO TESTING

UML is a design and modeling language for object-oriented software and is widely used as a de facto standard that provides techniques to model the software from different perspectives.

It supports facilities both for abstract high-level modeling and for more detailed low-level modeling.

It consists of a variety of graphical diagram types that can be extended for specific application areas.

It is associated with a formal textual language, OCL (Object Constraint Language). UML also provides support for model-based testing

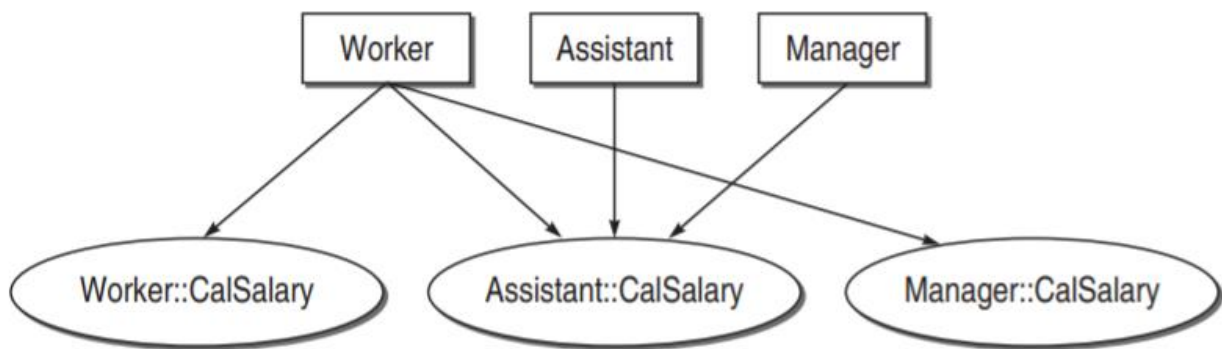


Figure 16.6 Balanced approach to implicit control flow

UML diagrams in software testing

The following UML diagrams are helpful in the testing activities mentioned below:

Use-case diagrams: testing of system-level functional requirements, acceptance testing

Class diagrams: class (module / unit) testing, integration testing

Sequence diagrams, collaboration diagrams: integration testing, testing of control and interaction between objects, testing of communication protocols between (distributed) objects

Activity diagrams: testing of work flow and synchronization within the system, white-box testing of control flow

State diagrams (state charts): state-based testing

Package diagrams, component diagrams: integration testing

Deployment diagrams: system testing

System Testing based on Use-Cases

Use-case In general, a use-case is a sequence of interactions by which the user accomplishes a task in a dialogue with the system.

use-case in testing, we follow the following:

- use-case = one particular way to use the system
- use-case (in testing) = user requirement
- set of all use-cases = complete functionality of the system
- set of all use-cases (in testing) = interface between the users (actors) and the system

Scenario It is an instance of a use-case, expressing a specific task, by a specific actor, at a specific time, and using specific data.

Use-case model This is a set of use-case diagrams, each associated with a textual description of the user's task. For testing, the use-case model must be extended with:

- The domain of each variable participating in the use-cases, „ the input/output relationships among the variables,
- The relative frequency of the use-cases,
- The sequential (partial) order among the use-cases.

use, case-based testing is a technique for generating test cases and recommended configurations for system-level testing. The testers build a test model based on the standard UML notions of use-cases, actors, and the relationships between these elements. The use-cases are enhanced with additional information, including inputs from actors, outputs to the actors, and how the use-case affects the state of the system.

To perform this testing, the tester needs to identify four things:

- the use-cases of interest, the actors involved in using the system,
- the input, output, and system effects for the use-cases,
- the flows of interest between the use-cases.

A use-case is a semantically meaningful function that provides some value from the user's point of view. For example, saving a file in a word processing system would be represented by the Save File use-case.

Web based Systems: Challenges in testing for web based software, quality aspects, web engineering, testing of web based systems, Testing mobile systems.

WEB-BASED SYSTEM

- The web-based software system consists of a set of web pages and components that interact to form a system which executes using web server(s), network ,HTTP, and a browser, and in which user input affects the state of the system.
- Thus, web-based systems are typical software programs that operate on the Internet, interacting with the user through an Internet browser.
- Some other terms regarding these systems are given below:

- **Web page** is the information that can be viewed in a single browser window. A website is a collection of web pages and associated software components that are related semantically by content and syntactically through links and other control mechanisms. Websites can be dynamic and interactive.
- **A web application** is a program that runs as a whole or in part on one or more web servers and that can be run by users through a website.
- Web applications require the presence of web server in simple configurations and multiple servers in more complex settings.
- Such applications are called **web based applications**. Similar applications, which may operate independent of any servers and rely on operating system services to perform their functions, are termed **web-enabled applications**.
- These days, with the integration of technologies used for the development of such applications, there is a thin line separating web-based and web-enabled applications, collectively refer to both of them as web applications.

CHALLENGES IN TESTING FOR WEB-BASED SOFTWARE

- web-based software are different as compared to traditional systems.
- The environment of their creation and then test them. Keeping in view the environment and behaviour of web-based systems,
- face many challenges while testing them. These challenges become issues and guidelines when we perform testing of these systems.
- Some of the challenges and quality issues for web-based system are discussed here:

Diversity and complexity

- Web applications interact with many components that run on diverse hardware and software platforms.
- They are written in diverse languages and they are based on different programming approaches such as procedural, OO, and hybrid languages such as Java Server Pages (JSPs).
- The client side includes browsers, HTML, embedded scripting languages, and applets.
- The server side includes CGI, JSPs, Java Servlets, and .NET technologies.
- They all interact with diverse back-end engines and other components that are found on the web server or other servers.
- **Dynamic environment** The key aspect of web applications is its dynamic nature. The dynamic aspects are caused by uncertainty in the program behaviour, changes in application requirements, rapidly evolving web technology itself, and other factors.
- The dynamic nature of web software creates challenges for the analysis, testing, and maintenance for these systems.
- For example, it is difficult to determine statically the application's control flow because the control flow is highly dependent on user input and sometimes in terms of trends in user behaviour over time or user location. Not knowing which page an application is likely to display hinders statically modeling the control flow with accuracy and efficiency.

Very short development time Clients of web-based systems impose very short development time, compared to other software or information systems projects. For e.g. an e-business system, sports website, etc.

Continuous evolution Demand for more functionality and capacity after the system has been designed and deployed to meet the extended scope and demands, i.e. scalability issues.

Compatibility and interoperability

- There may also be compatibility issues that make web testing a difficult task.
- Web applications often are affected by factors that may cause incompatibility and interoperability issues.
- The problem of incompatibility may exist on both the client as well as the server side. The server components can be distributed to different operating systems.
- Various versions of browsers running under a variety of operating systems can be there on the client side. Graphics and other objects on a website have to be tested on multiple browsers.
- If more than one browser will be supported, then the graphics have to be visually checked for differences in the physical appearance.
- The code that executes from the browser also has to be tested. There are different versions of HTML. They are similar in some ways but they have different tags which may produce different features.

QUALITY ASPECTS

some of the quality aspects which must be met while designing these systems are

1.Reliability

It is an expected feature that the system and servers are available at any time with the correct results. It means that the web systems must have high availability to be a reliable system.

If one web software fails, the customer should be able to switch to another one.

Thus, if web software is unreliable, websites that depend on the software will lose customers and the businesses may lose large amount of money.

2.Performance

Performance parameters for web systems must also be tested. If they are not tested, they may even lead to the total failure of the system. Performance testing is largely on the basis of load testing, response time, download time of a web page, or transactions performed per unit time.

3.Security

Since web systems are based on Internet, there are possibilities of any type of intrusion in these systems. Thus, security of web applications is another challenge for the web applications developers as well as for the testing team.

Security is critical for e-commerce websites.

Tests for security are often broken down into two categories:

testing the security of the infrastructure hosting the web application and testing for vulnerabilities of the web application. Some of the points that should be considered for infrastructure are firewalls and port scans. For vulnerabilities, there is user authentication, cookies, etc. Data collected must be secured internally. Users should not be able to browse through the directories in the server. A cookie is a text file on a user's system that identifies the user. Cookies must always be encrypted and must not be available to other users.

4.Usability

- This is another major concern while testing a web system. The users are real testers of these systems.
- If they feel comfortable in using them; then the systems are considered of high quality. Thus, usability also becomes an issue for the testers.
- The outlook of the system, navigation steps, all must be tested keeping the large categories of users in mind. Moreover, the users expect to be able to use the websites without any training.
- Thus, the software must flow according to the users' expectations, offer only needed information, and when needed, should provide navigation controls that are clear and obvious.

5.Scalability

- With the evolution of fast technology changes, the industry has developed new software languages, new design strategies, and new communication and data transfer protocols, to allow websites to grow as needed. Therefore, designing and building web software applications that can be easily scaled is currently one of the most interesting and important quality issues in software design.
- Scalability is defined as the web application's capacity to sustain the number of concurrent users and/or transactions, while sustaining sufficient response times to its users.
- To test scalability, web traffic loads must be determined in order to obtain the threshold requirement for scalability. Sometimes, existing traffic levels are used to simulate the load.

6.Availability

- Availability not only means that the web software is available at any time, but in today's environment it also must be available when accessed by diverse browsers.
- To be available in this sense, websites must adapt their presentations to work with all browsers, which require more knowledge and effort on the part of the developers. Maintainability Web-based software has a much faster update rate.
- Maintenance updates can be installed and immediately made available to customers through the website. Thus, even small individual changes (such as changing the label on a button) can be installed immediately.
- One result of this is that instead of maintenance cycles of months or years, websites have maintenance cycles of days or even hours.

WEB ENGINEERING (WEBE)

The diversity, complexity, and unique aspects of web-based software were not recognized and given due consideration by the developers early in the development of these systems.

These systems also lack proper testing and evaluation. As a result, they faced a web crisis or what is called a software crisis.

Web engineering is an application of scientific, engineering, and management principles and disciplined and systematic approaches to the successful development, deployment, and maintenance of high-quality web-based systems and applications.

Web engineering activities involve all the aspects starting from the conception and development to the implementation, performance evaluation, and continual maintenance.

1 ANALYSIS AND DESIGN OF WEB-BASED SYSTEMS

Various methods have been proposed in the recent years for modeling web based systems for analysis and design.

Keeping in consideration the nature of web applications, the design of these systems must consider the following:

- Design for usability—interface design, navigation.
- Design for comprehension.
- Design for performance—responsiveness.
- Design for security and integrity.
- Design for evolution, growth, and maintainability.
- Design for testability.
- Graphics and multimedia design.
- Web page development.

1. Conceptual Modeling

- The conceptual model is a static view of the system that shows a collection of static elements of the problem domain.
- To model this, class and association modeling elements of UML are used. Additionally, role names, multiplicities, different forms of associations supported by the UML like aggregation, inheritance, composition, and association class can also be used to improve the modeling.
- If there are many classes in conceptual modeling, then packages may also be used.
- Since the conceptual model will show the objects involved in typical activities, use-cases with activity diagrams are also used for conceptual modeling.

2. Navigation Modeling

- There are two types of navigation modeling:
- navigation space model and navigation structure model.
- The navigation space model specifies which objects can be visited by navigation through the application. It is a model at the analysis level.
- The navigation structure model defines how these objects are reached. It is a model at the design level.

- The navigation models are represented by stereotyped class diagrams consisting of classes of those objects which can be visited by navigation through the web application and the associations which specify which objects can be reached through navigation.
- The main modeling elements are the stereotyped class «navigation class» and the stereotyped association «direct navigability».
- However, navigation model considers only those classes of the conceptual model that are relevant for navigation. Further, the navigation space model can be enhanced.

3.Presentation Modeling

- This model transforms the navigation structure model in a set of models that show the static location of the objects visible to the user, i.e. a schematic representation of these objects.
- Using this presentation model, the discussion regarding interface design can be discussed with the customer.
- The sketches of these representations are drawn using a class diagram with UML composition notation for classes, i.e. containment represented by graphical nesting of the symbols of the parts within the symbol of the composite.

4.Web Scenarios Modeling

- The dynamic behaviour of any web scenarios including the navigation scenarios are modeled here.
- The scenarios help in introducing more specification details.
- The navigation structure model is detailed here showing all the sequences of the behaviour of the objects, i.e. stimuli that trigger the transitions and explicitly including the actions to be performed.
- This modeling is done with the help of sequence interaction diagram/collaboration diagrams and state chart diagrams.

5.Task Modeling

The tasks performed by the user or system are modeled here. Basically, the use-cases as tasks are refined here. Since use-cases are refined by activity diagrams in UML, these diagrams are used for task modeling.

- The modeling elements for task modeling are those for activity diagrams, i.e. activities, transitions, branches, etc.
- A task hierarchy can also be represented from coarse to fine-grained activities. The temporal order (with branches) between tasks is expressed by transitions between activities.
- Task modeling also comprises of the description of the objects—called referents—that the user will perceive. For this purpose, presentation and conceptual objects are included in the task model. The relationship between tasks and these objects is expressed as object flow visualized as directed dashed lines.
- The ingoing presentation objects express user input through these presentation objects and outgoing presentation objects express output to the user through these presentation objects. The conceptual objects express input and output of tasks.

6.Configuration Modeling

- Since web systems are diverse and complex in nature, we need to consider all the configuration and attributes that may be present on the client as well as on the server side.
- Moreover, some complex issues like deployment of process among multiple servers, compatibility with server OS, security issues, etc. are also considered.
- Deployment diagrams are used for configuration modeling.

2. DESIGN ACTIVITIES Based on the discussion above, the following design activities have been identified for a web-based system:

1. Interface Design

It considers the design of all interfaces recognized in the system including screen layouts. It also includes the types of navigation possible through the web system. All the interface design and navigational design issues are first discussed with the end-user. Interfaces should be designed such that they are flexible, consistent, and readable.

2.Content Design

All the contents regarding the web application identified as static elements in the conceptual modeling are designed here. The objects identified are known as content objects. Thus, these content objects and their relationships are designed here.

3.Architecture Design

The overall architecture of web applications is divided into the following three categories:

- (i) An overall system architecture describing how the network and the various servers (web servers, application servers, and database servers) interact.
- (ii) Content architecture design is needed to create a structure to organize the contents. Thus, this design is for structured presentation of the content objects. The content structure depends on factors like nature of the application, nature of the information, etc.
- (iii) Application architecture is designed within the context of the development environment in which the web application is developed. This design provides an infrastructure so that functioning of the application in the form of components is clearly understood.

4.Presentation Design

- This design is related to the look and feel of the application.
- The usability issues are considered here. The idea is that web application should be easy and flexible to use by its user.
- The design issues are related to layout of the web pages, graphics to be inserted, colour schemes, typefaces, etc.
- Navigation design Navigational paths are designed such that users are able to navigate from one place to another in the web application.
- The identified content objects, which are participating in navigation, become the navigation nodes and links between them define the navigation paths.

TESTING OF WEB-BASED SYSTEMS

Web-based systems have a different nature as compared to traditional systems.

The key differences between them. Due to the environment difference and challenges of dynamic behaviour, complexity and diversity also makes the testing of these systems a challenge.

They need to be tested for various quality parameters which are a must for these systems like security, usability, etc.

Test cases should be written covering the different scenarios not only of the functional usage but also the technical implementation environment conditions such as network speeds, screen resolution, etc.

For example, an application may work fine on Broadband Internet users but may perform miserably for users with dial-up connections.

Web applications are known to give errors on slow networks, whereas they perform well on high-speed connections.

Web pages don't render correctly for certain situations but work fine with others.

Testing should be performed keeping in view all the possible categories of users in view. This becomes the issues in usability testing.

Another issue of great concern is the security testing of web applications.

There are two cases. For Intranet-based applications, there are no such threats on the application.

The strategy for testing web-based systems is the same as for other systems, i.e. verification and validation.

1 INTERFACE TESTING

2 USABILITY TESTING

3 CONTENT TESTING

4 NAVIGATION TESTING

5 CONFIGURATION/COMPATIBILITY TESTING

6 SECURITY TESTING

7 PERFORMANCE TESTING

1 INTERFACE TESTING

1. Interface is a major requirement in any web application.
2. More importantly, the user interface with web application must be proper and flexible.

3. Therefore as a part of verification, present model and web scenarios model must be checked to ensure all interfaces.
4. The interfaces between the concerned client and servers should also be considered.
5. There are two main interfaces on the server side:
6. web server and application server interface and application server and database server interface.
7. Web applications establish links between the web server and the application server at the onset.
8. Testing should check for appropriate error messages, roll back in case of failure to execute or user interruption.

2 USABILITY TESTING

1. The presentation design emphasizing the interface between user and web application gives rise to usability testing.
2. The actual user of application should feel good while using the application and understand every thing visible to him on it.
3. Usability testing is not a functionality testing, but the web application is reviewed and tested from a user's viewpoint.
4. The importance of usability testing can be realized with the fact that we can even lose users because of a poor design.
5. For example, check that form controls, such as boxes and buttons, are easy to use, appropriate to the task, and provide easy navigation for the user.
6. The critical point for designers and testers in this attesting is that web application must be as pleasant and flexible as possible to the user.
7. Usability testing may include tests for navigation.
8. For verification of web application, the presentation design must be checked properly so that most of the errors are resolved at the earlier stages only.
9. Verification can be done with a technique called card-sorting technique The actual user session logs can be recorded and evaluated from the usability point.

The general guidelines for usability testing are:

1. Present information in a natural and logical order.
2. Indicate similar concepts through identical terminology and graphics. Adhere to uniform conventions for layout, formatting, typefaces, labeling, etc.
3. Do not force users to remember key information across documents.
4. Keep in consideration that users may be from diverse categories with various goals. Provide understandable instructions where useful. Lay out screens in such a manner that frequently accessed information is easily found.
5. The user should not get irritated while navigating through the web application. Create visually pleasing displays. Eliminate information which is irrelevant or distracting.
6. Content writer should not mix the topics of information. There should be clarity in the information being displayed.

7. Organize information hierarchically, with more general information appearing before more specific detail. Encourage the user to delve as deeply as needed, but to stop whenever sufficient information has been received.

8. Check that the links are active such that there are no erroneous or misleading links.

3 CONTENT TESTING

- The content seen on the web pages has a strong impression on its user.
- If these contents are not satisfactory to him, he may not visit the web page again.
- Check the completeness and correctness properties of web application content.
- Check that certain information is available on a given web page, links between pages exist, or even check the existence of the web pages themselves (completeness property).
- Check for the default values of fields and also wrong inputs to the fields in the forms. Options to create forms if any, form delete, view, or modify the forms must also be checked.

Static testing may consider checking the following points:

1. Various layouts.
2. Check forms for their field validation, error message for wrong input, optional and mandatory fields with specified length, buttons on the form, etc.
3. A table is present and has the expected number of rows and columns and pre-defined properties.
4. Grammatical mistakes in text description of web page.
5. Typographical mistakes. 6. Content organization. 7. Content consistency.
8. Data integrity and errors while you edit, delete, modify the forms.
9. Content accuracy and completeness.
10. Relationship between content objects.
11. Text contents.
12. Text fragments against formatting expectations. This differs slightly from simple text checking in that the formatting tags can be located loosely on the page as opposed to a fixed string for text content.
13. Graphics content with proper visibility.
14. Media contents to be placed at appropriate places.
15. All types of navigation links like internal links, external links, mail links, broken links to be placed at appropriate places.
16. All links on a web page are active.

There may also be dynamic contents on a web page. Largely, dynamic testing will be suitable in testing these dynamic contents.

These dynamic contents can be in many forms.

One possibility is that there are constantly changing contents, e.g. weather information web pages or online newspaper.

Another case may be that web applications are generated dynamically from information contained in a database or in a cookie.

Many web applications today work interactively in the manner that in response to a user request for some information, it interacts with some DBMS, extracts the relevant data, creates the dynamic content objects for this extracted data, and sends these content objects to the user for display.

4 NAVIGATION TESTING

checked the navigation contents in interface testing.

But to ensure the functioning of correct sequence of those navigations, navigation testing is performed on various possible paths in the web application.

Design the test cases such that the following navigations are correctly executing:

1. Internal links
2. External links
3. Redirected links
4. Navigation for searching inside the web application

The errors must be checked during navigation testing for the following:

1. The links should not be broken due to any reason.
2. The redirected links should be with proper messages displayed to the user.
3. Check that all possible navigation paths are active.
4. Check that all possible navigation paths are relevant.
5. Check the navigations for the back and forward buttons, whether they are working properly.

5 CONFIGURATION/COMPATIBILITY TESTING

- Diversity in configuration for web applications makes the testing of these systems very difficult.
- various types of browsers supporting different operating systems, variations in servers, networks, etc. Therefore, configuration testing becomes important so that there is compatibility between various available resources and application software.
- The testers must consider these configurations and compatibility issues so that they can design the test cases incorporating all the configurations.

Some points to be careful about while testing configuration are:

1. There are a number of different browsers and browser options. The web application has to be designed to be compatible for majority of the browsers.
2. The graphics and other objects on a website have to be tested on multiple browsers. If more than one browser will be supported, then the graphics have to be visually checked for differences in the physical appearance. Some of the things to check are centering of objects, table layouts, colours, monitor resolution, forms, and buttons.
3. The code that executes from the browser also has to be tested. There are different versions of HTML. They are similar in some ways but they have different tags which may produce different features. Some of the other codes to be tested are Java, JavaScript, ActiveX, VBscripts, CgiBin Scripts, and Database access. Cgi-Bin Scripts have to be checked for end-to-end operations and is most essential for e-commerce sites. The same goes for database access.
4. All new technologies used in the web development like graphics designs, interface calls like different API's, may not be available in all the operating systems. Test your web application on different operating systems like Windows, Unix, MAC, Linux, Solaris with different OS flavors.

6 SECURITY TESTING

The Web applications store more vital data and the number of transactions on the web has increased tremendously with the increasing number of users.

Therefore, in the Internet environment, the most challenging issue is to protect the web applications from hackers, crackers, spoofers , virus launchers, etc.

Through security testing, try to ensure that data on the web applications remain confidential, i.e. there is no unauthorized access.

Security testing also ensures that users can perform only those tasks that they are authorized to perform.

Security Test Plan

Security testing can be divided into **two categories**:

1. Testing the security of the infrastructure hosting the web application and testing for vulnerabilities of the web application.
2. Firewalls and port scans can be the solution for security of infrastructure.

For vulnerabilities, user authentication, restricted and encrypted use of cookies, data communicated must be planned.

Planning for security testing can be done with the help of some threat models.

These models may be prepared at the time of requirement gathering and test plan can also be prepared correspondingly.

These threat models will help in incorporating security issues in designing and later can also help in security testing.

Various Threat Types and their corresponding Test cases

1.Unauthorized user/Fake identity/Password cracking

When an unauthorized user tries to access the software by using fake identity, then security testing should be done such that any unauthorized user is not able to see the contents/ data in the software.

2.Cross-site scripting (XSS)

When a user inserts HTML/client-side script in the user interface of a web application and this insertion is visible to other users, it is called cross-site scripting (XSS).

Attacker can use this method to execute malicious script or URL on the victim's browser.

Using cross-site scripting, attacker can use scripts like JavaScript to steal user cookies and information stored in the cookies. To avoid this, the tester should additionally check the web application for XSS.

3.Buffer overflows

Buffer overflow is another big problem when handling memory allocation if there is no overflow check on the client software.

Due to this problem, malicious code can be executed by the hackers. In the application, check the buffer overflow module and the different ways of submitting a range of lengths to the application.

4.URL manipulation

There may be chances that communication through HTTP is also not safe.

The web application uses the HTTP GET method to pass information between the client and the server.

The information is passed in parameters in the query string. Again, the attacker may change some information in query string passed from GET request so that he may get some information or corrupt the data. When one attempts to modify the data, it is known as fiddling of data.

The tester should check if the application passes important information in the query string and design the test cases correspondingly.

Write the test cases such that a general user tries to modify the private information.

5.SQL injection

Hackers can also put some SQL statements through the web application user interface into some queries meant for querying the database.

In this way, he can get vital information from the server database. Even if the attacker is successful in crashing the application, from the SQL query error shown on the browser, the attacker can get the information they are looking for.

Design the test cases such that special characters from user inputs should be handled/escaped properly in such cases.

6.Denial of service

When a service does not respond, it is denial of service. There are several ways that can make an application fail. For example, heavy load put on the application, distorted data that may crash the application, overloading of memory, etc. Design the test cases considering all these factors.

7 PERFORMANCE TESTING

Performance testing helps the developer to identify the bottlenecks in the system and can be rectified.

In performance testing, we evaluate metrics like response time, throughput, and resource utilization against desired values.

Using the results of this evaluation, we are able to predict whether the software is in a condition to be released or requires improvement before it is released.

Moreover, we can also find the bottlenecks in the web application. Bottlenecks for web applications can be code, database, network, peripheral devices, etc.

Performance Parameters Performance parameters, against which the testing can be performed, are given here:

Resource utilization The percentage of time a resource (CPU, Memory, I/O, Peripheral, Network) is busy.

Throughput The number of event responses that have been completed over a given interval of time.

Response time The time lapsed between a request and its reply.

Database load The number of times database is accessed by web application over a given interval of time.

Scalability The ability of an application to handle additional workload, without adversely affecting performance, by adding resources such as processor, memory, and storage capacity.

Round-trip time How long does the entire user-requested transaction take, including connection and processing time?

Types of Performance

Testing Performance tests are broadly divided into following categories:

1. Load testing

The site should handle many simultaneous user requests, large input data from users, simultaneous connection to DB, heavy load on specific pages, etc.

It focuses on determining or validating performance characteristics of the system when subjected to workloads and load volumes anticipated during production operations.

It refers to how much load (the best example of load in web application is how many concurrent users) you can put on the web application and it will still serve flawlessly.

There are a few types of load testing which we can be performed.

perform capacity testing to determine the maximum load the web service can handle before failing. Capacity testing reveals the web services' ultimate limit.

perform scalability testing to determine how effectively the web service will expand to accommodate an increasing load.

2.Stress testing

stress refers to stretching the system beyond its specification limits.

Web stress testing is performed to break the site by giving stress and check how the system reacts to stress and how the system recovers from crashes.

It focuses on determining or validating performance characteristics of the system when subjected to conditions beyond those anticipated during production operations.

It also tests the performance of the system under stressful conditions, such as limited memory, insufficient disk space, or server failure.

These tests are designed to determine under what conditions an application will fail, how it will fail, and how gracefully it may recover from the failure.

Some examples of graceful failure are: the system saves state at the time of failure and does not crash suddenly; On restarting it, the system recovers from the last good state; the system shows meaningful error messages to the user.

TESTING MOBILE SYSTEMS:

Mobile application testing is a process by which application software developed for hand held mobile devices is tested for its functionality, usability and consistency. Mobile application testing can be automated or manual type of testing. Mobile applications either come pre-installed or can be installed from mobile software distribution platforms. Mobile devices have witnessed a phenomenal growth in the past few years.

Key Challenges in Mobile Application Testing:

1. **Variety of Mobile Devices:** Mobile devices differ in screen sizes, input methods (QWERTY, touch, normal) with different hardware capabilities.
2. **Diversity in Mobile Platforms/OSes:** There are different mobile operating systems in the market. The major ones are Android, iOS, Symbian, Windows Phone, and BlackBerry(RIM). Each operating system has its own limitations. Testing a single application across multiple devices running on the same platform and every platform poses a unique challenge for testers.
3. **Device Availability:** Access to the right set of devices when there is an ever-growing list of devices and operating system versions is a constant mobile application testing challenge. Access to devices can become even more challenging if testers are spread across different locations.
4. **Mobile network operators:** There are over 400 mobile network operators in the world ;out of which some are CDMA, some GSM.
5. **Scripting:** The variety of devices makes executing a test script (scripting) a key challenge. As devices differ in keystrokes, input methods, menu structure and display properties single script does

not function on every device.

6. **Choosing how to test:** There are two main ways of testing mobile applications: testing on real devices or testing on emulators. Unfortunately, neither method is flawless.[5] Emulators often miss issues that can only be caught by testing on real devices, but because of the multitude of different devices in the market, real devices can be expensive to purchase and time-consuming to use for testing.

Types of Mobile Application Testing:

1. **Functional Testing:** Functional testing ensures that the application is working as per the requirements. Most of the test conducted for this is driven by the user interface and call flows.

2. **Laboratory Testing:** Laboratory testing, usually carried out by network carriers, is done by simulating the complete wireless network. This test is performed to find out any glitches when a mobile application uses voice and/or data connection to perform some functions.

3. **Performance Testing:** This testing process is undertaken to check the performance and behavior of the application under certain conditions such as low battery, bad network coverage, low available memory, simultaneous access to application's server by several users and other conditions.

4. **Memory Leakage Testing:** Memory leakage happens when a computer program or application is unable to manage the memory it is allocated resulting in poor performance of the application and the overall slowdown of the system. As mobile devices have significant constraints of available memory, memory leakage testing is crucial for the proper functioning of an application

5. **Interrupt Testing:** An application while functioning may face several interruptions like incoming calls or network coverage outage and recovery. The different types of interruptions are:

- ☐ Incoming and Outgoing SMS and MMS
- ☐ Incoming and Outgoing calls
- ☐ Incoming Notifications
- ☐ Battery Removal
- ☐ Cable Insertion and Removal for data transfer

6. **Usability testing:** Usability testing is carried out to verify if the application is achieving its goals and getting a favorable response from users. This is important as the usability of an application is its key to commercial success (it is nothing but user friendliness).

7. **Installation testing:** Certain mobile applications come pre-installed on the device whereas others have to be installed from the store. Installation testing verifies that the installation process goes smoothly without the user having to face any difficulty. This testing process covers installation updating and uninstalling of an application